LOCAL SETUP · DOCKER DESKTOP

Kubernetes on Your *Local Machine*

A complete guide to understanding and running Kubernetes locally using Docker Desktop. Covers every core component with examples from a real MERN stack project.

Docker Desktop | MERN Stack | nginx Ingress | Node.js · Express

00 What is Kubernetes?

The problem it solves and why you need it

CORE CONCEPT

Kubernetes (K8s) is a container orchestration system. It manages how your Docker containers run — making sure the right number are running, restarting them when they crash, routing traffic between them, and scaling them up when load increases. You describe what you want in YAML files and Kubernetes makes it happen.

Without vs With Kubernetes

Situation	Without Kubernetes	With Kubernetes
Container crashes	App goes down, manual restart needed	Kubernetes restarts it automatically

Situation	Without Kubernetes	With Kubernetes
Traffic spike	App slows down or crashes	HPA adds more containers automatically
Multiple services	Manual ports, complex networking	Internal DNS — services talk by name
Deploy new version	Downtime during update	Rolling update — zero downtime
Multiple replicas	Manage each container manually	Set replicas: 3 and Kubernetes handles the rest

Key Terms at a glance

TERM	SIMPLE MEANING
`Pod`	Smallest unit. One or more containers running together.
`Deployment`	Manages pods — keeps them running, handles updates and rollbacks.
`Service`	Stable network endpoint that finds pods using labels and routes traffic to them.
`Ingress`	HTTP routing rules — which domain/path goes to which service.
`Ingress Controller`	The actual pod (nginx) that reads Ingress rules and routes real traffic.
`Namespace`	Virtual cluster inside a cluster. Used to separate environments.
`ConfigMap`	Store non-secret config (env variables) separately from your image.

TERM	SIMPLE MEANING
Secret	Store sensitive config (passwords, tokens) encoded in the cluster.

01 Architecture

How all components fit together

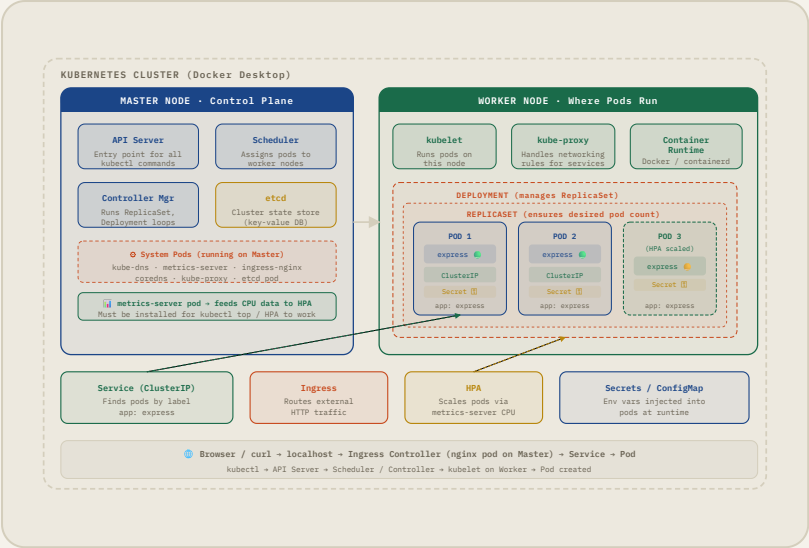
KEY CONCEPT

A Kubernetes cluster has two parts: the **Control Plane** (the brain — decides what runs where) and **Worker Nodes** (the muscle — where your pods actually run). On Docker Desktop both live on your local machine.

Full traffic flow

- **Browser / Client**
  Makes HTTP request to express.local or localhost
- **Ingress Controller (nginx pod)**
  Receives traffic. Reads Ingress rules to decide where to send it.
- **Ingress Resource (your ingress.yaml rules)**
  Defines which host/path maps to which Service
- **Service (ClusterIP)**
  Finds pods using label selector. Load balances across all matching pods.
- **Pod (your Express container)**
  Handles the request and sends back a response

Cluster Architecture Diagram



Project file structure

PROJECT LAYOUT

```
your-project/
├── Backend/
│   ├── server.js
│   ├── package.json
│   └── dockerfile
└── k8s/                    # all Kubernetes YAML file
    ├── deployment.yaml     # how to run the app
    ├── service.yaml        # how to reach the app
    └── ingress.yaml        # how external traffic
```

02 Pod

The smallest deployable unit in Kubernetes

KEY CONCEPT

A **Pod** is a wrapper around one or more containers. All containers in a pod share the same network (same IP address) and storage. In practice, most pods contain a single container — your app. You rarely create pods directly — a Deployment creates and manages them for you.

What a pod contains

WHAT	DESCRIPTION
`Container(s)`	One or more Docker containers. They share the same localhost inside the pod.
`IP Address`	Each pod gets its own IP inside the cluster. This IP changes every time a pod restarts — which is why you use a Service instead of the pod IP directly.
`Labels`	Key-value tags like app: express. Services and Deployments use these to find the pod.
`Resources`	CPU and memory limits and requests defined in the pod spec.

Pod lifecycle

```
Pending → pod is scheduled, image is being pulled
    ↓
Running → container started successfully
    ↓
Ready → readiness probe passed, pod receives
traffic ✅
    ↓
CrashLoopBackOff → container keeps crashing,
Kubernetes retries
    ↓
ImagePullBackOff → cannot pull the Docker image
```

💡 You **never create pods directly** in production. Always use a Deployment. If a pod is created directly and crashes, nothing restarts it. A Deployment automatically replaces crashed pods.

Check pod status

TERMINAL copy

```
kubectl get pods                          # list all
kubectl describe pod <pod-name>           # full de
kubectl logs <pod-name>                   # app cons
kubectl exec -it <pod-name> -- sh         # shell ins
```

03 Deployment

Manages pods — runs, scales, updates and rolls back

KEY CONCEPT

A **Deployment** is a manager for your pods. You tell it what image to run, how many replicas you want, and what resources each pod needs. It ensures that exact state is always maintained — if a pod crashes, Deployment creates a new one. If you update the image, it does a rolling update with zero downtime.

What Deployment is responsible for

RESPONSIBILITY	WHAT IT MEANS
`Replica management`	Always keeps the exact number of pods running. If one crashes, a new one is created immediately.
`Rolling update`	When you push a new image, Kubernetes replaces pods one by one. Old pods serve traffic until new ones are ready — zero downtime.
`Rollback`	If a new version has a bug, one command goes back to the previous working version.
`Scaling`	Change replica count manually or let HPA do it automatically based on CPU.

Full deployment.yaml with explanation

DEPLOYMENT.YAML

```yaml
apiVersion: apps/v1          # Deployment lives i
kind: Deployment
metadata:
  name: express-deployment
spec:
  replicas: 2              # always keep 2 pods r
  selector:
    matchLabels:
      app: express          # (1) manage pods tha
  template:               # blueprint for each pod
    metadata:
      labels:
        app: express        # (2) stick this labe
        owner: ankur       # extra label — deploy
    spec:
      containers:
        - name: express
          image: express-k8s:latest
          imagePullPolicy: IfNotPresent  # use lo
          ports:
            - containerPort: 3000         # port
          resources:
            limits:
              memory: "128Mi"            # max me
              cpu: "500m"               # max CPU
            requests:
              memory: "64Mi"             # memory
              cpu: "250m"               # CPU res
```

Key fields explained

FIELD	VALUE	WHAT IT DOES
apiVersion	apps/v1	Deployment belongs to the apps API group. Not the core group (v1) — that's for Pods and Services.
replicas	2	Always maintain 2 running pods. If one crashes, a new one is created immediately.

FIELD	VALUE	WHAT IT DOES
selector.matchLabels	app: express	The Deployment manages pods that have ALL these labels. Must match template.metadata.labels.
template.metadata.labels	app: express	Labels stamped on every pod this Deployment creates. Service uses these to find pods.
imagePullPolicy	IfNotPresent	Use local image if available. On EKS change this to Always so it pulls from ECR on every restart.
resources.requests	cpu: 250m	CPU/memory reserved for this pod. HPA requires requests to be set — it uses this to calculate utilization %.
resources.limits	cpu: 500m	Maximum CPU/memory the pod can use. Prevents one pod from starving others.

CPU units explained

VALUE	MEANING
1000m	1 full CPU core
500m	Half a CPU core
250m	Quarter of a CPU core

Useful Deployment commands

TERMINAL copy

```
kubectl get deployments
kubectl rollout status deployment/express-deploym
kubectl rollout undo deployment/express-deploymen
```

```
kubectl scale deployment express-deployment --rep
kubectl rollout restart deployment/express-deploy
```

04 ReplicaSet

Ensures a fixed number of identical pods are always running

KEY CONCEPT

A **ReplicaSet** is the component that guarantees *N* copies of your pod are always running. If a pod crashes, ReplicaSet creates a replacement. If you have too many pods, it deletes extras. You almost never create a ReplicaSet directly — a **Deployment creates and manages a ReplicaSet for you**, and also handles rolling updates and rollbacks on top of it.

Deployment → ReplicaSet → Pods relationship

Deployment → owns and manages a ReplicaSet
↓
ReplicaSet → ensures *replicas: 2* pods are always alive
↓
Pod 1 (running ✅) Pod 2 (running ✅)
↓ Pod 1 crashes
ReplicaSet detects: 1 pod running, desired is 2
↓
ReplicaSet creates Pod 3 automatically ✅

Why you don't write ReplicaSet YAML directly

IF YOU USE	WHAT YOU GET
`Deployment`	ReplicaSet management + rolling updates + rollback history. Always use this in practice.
`ReplicaSet directly`	Pod count maintenance only. No rolling updates, no rollback. Not recommended.

💡 When you run `kubectl get replicasets` you'll see an auto-generated ReplicaSet like `express-deployment-7d9f8b6c4` — the hash suffix is a fingerprint of the pod template. When you update the image, a **new ReplicaSet** is created and the old one is scaled down to zero (but kept for rollback).

Viewing ReplicaSets

TERMINAL copy

```
kubectl get replicasets
kubectl describe replicaset <rs-name>          #

# After a rollout you will see TWO replicasets:
# express-deployment-7d9f8b6c4   2         2
# express-deployment-5c8a3d1b2   0         0
```

How ReplicaSet selects pods

ReplicaSet uses **label selectors** — it looks for pods with matching labels and counts them. If you manually create a pod with the same labels, the ReplicaSet will count it and may delete one of its own pods to maintain the desired count. This is why labels must be unique per Deployment.

REPLICASET.YAML — FOR REFERENCE ONLY, USE DEPLOYMENT IN PRACTICE

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
```

```
  name: express-rs
spec:
  replicas: 2                  # desired pod coun
  selector:
    matchLabels:
      app: express             # manage pods with
  template:
    metadata:
      labels:
        app: express
    spec:
      containers:
        - name: express
          image: express-k8s:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 3000
```

04 Service

Stable network endpoint that routes traffic to pods

KEY CONCEPT

Pods are temporary — they crash and restart with new IP addresses. A **Service** gives you a stable endpoint that never changes. It finds pods using **label selectors** and load balances traffic across all matching pods automatically. When a pod is added or removed, the Service updates its list instantly.

How Service finds pods (Labels & Selectors)

SERVICE.YAML

```
apiVersion: v1               # Service is a core r
kind: Service
metadata:
  name: express-service
spec:
```

```
  selector:
    app: express            # find ALL pods with
  ports:
    - protocol: TCP
      port: 80              # port the service li
      targetPort: 3000      # port your app liste
  type: ClusterIP          # only accessible insi
```

Three Service types

Type	Accessible from	Use case
ClusterIP	Inside cluster only	Pod-to-pod communication. Default type. Used with Ingress.
NodePort	External via port number	Quick local testing. Access via localhost:30001. No Ingress needed.
LoadBalancer	External via cloud LB	Production on AWS/GCP. Creates real Load Balancer. Costs money.

port vs targetPort

FIELD	VALUE	MEANING
`port`	`80`	The port the Service exposes inside the cluster. Other pods call this service on port 80.
`targetPort`	`3000`	The port your app is actually listening on inside the container. Must match containerPort in deployment.

NodePort for local testing

SERVICE.YAML (NODEPORT)

```yaml
type: NodePort
ports:
  - port: 80
    targetPort: 3000
    nodePort: 30001    # access via localhost:300
```

How Service load balances

```
Service receives request (selector: app=express)
    ↓
Looks up Endpoints list (live pod IPs)
    ↓
Request 1 → Pod 1 (192.168.1.10)
Request 2 → Pod 2 (192.168.1.11)
Request 3 → Pod 3 (192.168.1.12)
Request 4 → Pod 1 (round robin) ✅
```

TERMINAL copy

```
kubectl get services
kubectl get endpoints express-service    # see whi
```

05 Ingress

HTTP routing rules — host and path based traffic routing

KEY CONCEPT

An **Ingress** resource is a set of HTTP routing rules written in YAML. It says things like: "traffic for `express.local/` should go to `express-service`" or "traffic for `express.local/auth` should go to `auth-service`". The Ingress itself does nothing — it only

> works when an **Ingress Controller** is installed to read and act on these rules.

Full ingress.yaml with explanation

INGRESS.YAML

```yaml
apiVersion: networking.k8s.io/v1   # Ingress is i
kind: Ingress
metadata:
  name: express-ingress
spec:
  ingressClassName: nginx          # which control
  rules:
    - host: express.local           # domain to ma
      http:
        paths:
          - path: /
            pathType: Prefix          # match / an
            backend:
              service:
                name: express-service
                port:
                  number: 80
```

Path based routing — multiple services on one domain

INGRESS.YAML — MULTI SERVICE

```yaml
rules:
  - host: express.local
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: main-service        # express.
              port: { number: 80 }
        - path: /auth
```

```
        pathType: Prefix
        backend:
          service:
            name: auth-service        # express.
            port: { number: 80 }
```

Key fields explained

FIELD	VALUE	WHAT IT DOES
`ingressClassName`	`nginx`	Tells Kubernetes which Ingress Controller should handle this rule. If you have multiple controllers (nginx, traefik), each only picks up rules meant for it.
`host`	`express.local`	Only traffic with this Host header matches this rule. Add it to /etc/hosts for local development.
`pathType: Prefix`	`Prefix`	Matches the path and everything after it. /api matches /api/users, /api/data etc.
`pathType: Exact`	`Exact`	Only matches that exact path. /api does NOT match /api/users.

For local development — skip the host

INGRESS.YAML — NO HOST (EASIEST)

```
rules:
  - http:                          # no host: field —
      paths:
        - path: /
          pathType: Prefix
```

```
      backend:
        service:
          name: express-service
          port:
            number: 80
```

Access via `http://localhost` directly — no /etc/hosts edit needed.

06 Ingress Controller

The actual nginx pod that handles real traffic

> KEY CONCEPT
>
> The **Ingress Controller** is a pod running nginx inside your cluster. It watches all Ingress resources and configures itself to route traffic according to those rules. Without the controller, your ingress.yaml is just a config file sitting there doing nothing. The controller is what makes it actually work.

Ingress vs Ingress Controller

What	Ingress Resource	Ingress Controller
What it is	A YAML file with routing rules	An actual nginx pod running in your cluster
Created by	You — kubectl apply -f ingress.yaml	You install once — kubectl apply -f nginx-url
Does it route traffic?	No — it only defines rules	Yes — reads rules and routes real HTTP traffic

What	Ingress Resource	Ingress Controller
Namespace	default (your app namespace)	ingress-nginx (its own namespace)

INSTALL NGINX INGRESS CONTROLLER

TERMINAL copy

```
kubectl apply -f \
  https://raw.githubusercontent.com/kubernetes/in
```

VERIFY CONTROLLER IS RUNNING

TERMINAL copy

```
kubectl get pods -n ingress-nginx

# Expected output:
NAME                                        READY
ingress-nginx-controller-xxxxxxxxx-xxxxx    1/1
```

Add express.local to /etc/hosts (Mac)

TERMINAL copy

```
sudo nano /etc/hosts

# Add this line at the bottom:
127.0.0.1  express.local
```

Now `http://express.local` in your browser will route to your nginx Ingress Controller on localhost.

💡 **ingressClassName: nginx** in your ingress.yaml is how the controller knows which rules are meant for it. If you had traefik installed too, it only picks up rules with `ingressClassName: traefik`.

07 Enable Kubernetes on Docker Desktop

One-click setup — no extra tools needed

STEP 1 — ENABLE IN DOCKER DESKTOP SETTINGS

Open Docker Desktop → click the **gear icon** (Settings) → click **Kubernetes** in the left sidebar → check **Enable Kubernetes** → click **Apply & Restart**.

Wait 2–3 minutes. When the bottom status bar shows a green Kubernetes icon, it is ready.

STEP 2 — VERIFY KUBECTL IS WORKING

TERMINAL copy

```
kubectl version
kubectl get nodes

# Expected output:
NAME             STATUS   ROLES
docker-desktop   Ready    control-plane ✅
```

STEP 3 — CHECK CURRENT CONTEXT

TERMINAL copy

```
kubectl config current-context
# Should return: docker-desktop

kubectl config get-contexts
# Lists all clusters kubectl knows about
```

💡 The `~/.kube/config` file stores all cluster connection info. When you later create an EKS cluster with eksctl, it adds a new entry here and switches the current context to EKS automatically. Switch back with `kubectl config use-context docker-desktop`.

08 Writing YAML Files

Structure, apiVersion, and how to combine files

KEY CONCEPT

Every Kubernetes resource is defined in a YAML file. Each file must have **apiVersion**, **kind**, **metadata**, and **spec**. The `apiVersion` tells Kubernetes which API group the resource belongs to. Different resources live in different groups.

apiVersion — which API group

RESOURCE	APIVERSION	WHY
Pod, Service, ConfigMap, Secret	v1	Core group — the oldest, most fundamental Kubernetes resources. No group prefix needed.
Deployment, StatefulSet, DaemonSet	apps/v1	Apps group — higher-level workload resources that manage pods.
Ingress, NetworkPolicy	networking.k8s.io/v1	Networking group — resources

RESOURCE	APIVERSION	WHY
		that manage cluster networking.
HorizontalPodAutoscaler	autoscaling/v2	Autoscaling group — resources that control scaling behavior.

Putting multiple resources in one file

COMBINED.YAML

```yaml
apiVersion: apps/v1
kind: Deployment
# ... deployment config

---                    # separator — new resource

apiVersion: v1
kind: Service
# ... service config
```

💡 Keep files **separate** as your project grows. One file per resource makes it easier to find and edit things. Use `kubectl apply -f k8s/` to apply the entire folder at once.

09 Labels & Selectors

How Deployments manage pods and Services find them

KEY CONCEPT

Labels are key-value tags on pods. **Selectors** are filters used by Deployments and Services to find pods with

specific labels. This is how all three resources connect — the Deployment owns pods via selector, the Service routes to pods via selector. No hardcoded pod names or IPs anywhere.

The three-way connection

HOW THEY CONNECT

```yaml
# deployment.yaml
selector:
  matchLabels:
    app: express       # (1) Deployment manages po
template:
  metadata:
    labels:
      app: express     # (2) Pod gets stamped with

---
# service.yaml
selector:
  app: express         # (3) Service routes to pod
```

Selector matching rules

POD LABELS	SELECTOR: APP=EXPRESS	RESULT
`app: express`	Matches	SELECTED ✅
`app: express, env: prod`	Matches (extra label ignored)	SELECTED ✅
`app: backend`	Does not match	IGNORED ❌
`app: express, env: staging`	Partially matches selector app=express only	SELECTED ✅

Multiple deployments using labels

USING ENV LABEL TO SEPARATE DEPLOYMENTS

```yaml
# staging-deployment.yaml
selector:
  matchLabels:
    app: express
    env: staging     # only manages staging pods

# production-deployment.yaml
selector:
  matchLabels:
    app: express
    env: production  # only manages production po
```

Both deployments exist in the same cluster but never interfere with each other because their selectors are different.

10 Deploy Your App

Build image, apply YAML, access in browser

STEP 1 — BUILD DOCKER IMAGE

TERMINAL copy

```
cd Backend/
docker build -t express-k8s:latest .
```

STEP 2 — APPLY ALL YAML FILES

TERMINAL copy

```
kubectl apply -f k8s/    # applies all files in t

# Or apply individually:
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
```

STEP 3 — VERIFY EVERYTHING IS RUNNING

TERMINAL copy

```
kubectl get pods          # STATUS: Running, REA
kubectl get services      # service exists with
kubectl get ingress       # ingress has an ADDRE
```

STEP 4 — ACCESS IN BROWSER

SETUP	URL
No host in ingress	http://localhost
host: express.local (with /etc/hosts)	http://express.local
NodePort service	http://localhost:30001

Update your app

TERMINAL copy

```
# 1. Rebuild image after code changes
docker build -t express-k8s:latest .

# 2. Restart deployment to use new image
kubectl rollout restart deployment/express-deploy
```

11 Debug Commands

Finding and fixing problems in your cluster

Common errors and what to do

ERROR	CAUSE	DEBUG COMMAND
`ImagePullBackOff`	Cannot pull Docker image. Usually local image not built or wrong name.	`kubectl describe pod <name>` → check Events section
`CrashLoopBackOff`	Container keeps crashing. App has a startup error.	`kubectl logs <pod-name>` → see app error output
`503 Service Unavailable`	Ingress controller running but no pods found. Label mismatch.	`kubectl get endpoints <service>` → if <none> labels don't match
`404 Not Found`	nginx running but no matching ingress rule for that host/path.	`kubectl describe ingress <name>` → check rules
`Pending (pod)`	No node has enough CPU/memory to schedule the pod.	`kubectl describe pod <name>` → check Events
`ECONNREFUSED`	Pod calling another service using localhost instead of service name.	Use `http://service-name` not `http://localhost:PORT`

Essential debug commands

TERMINAL copy

```
# Check what is running
kubectl get pods
```

```
kubectl get services
kubectl get ingress
kubectl get all                                  # ev

# Deep inspect
kubectl describe pod <pod-name>                  # ev
kubectl describe ingress <ingress-name>          # ro
kubectl describe service <service-name>          # se

# Logs
kubectl logs <pod-name>                          # ap
kubectl logs <pod-name> -f                       # fol
kubectl logs <pod-name> --previous               # log

# Network debug
kubectl get endpoints <service-name>             # pod
kubectl exec -it <pod-name> -- sh                # she
# Inside pod: test another service
wget -qO- http://some-service/api/data
```

12 Autoscaling (HPA)

Automatically add and remove pods based on CPU usage

KEY CONCEPT

The **HorizontalPodAutoscaler (HPA)** watches CPU usage across your pods and automatically scales the replica count up or down. When CPU exceeds your threshold it adds pods. When traffic drops it removes them. It requires **metrics-server** to read CPU data — on Docker Desktop this is *not* pre-installed and must be added manually.

Step 0 — Install Metrics Server (required for HPA)

⚠️ **HPA will not work without metrics-server.** Running `kubectl top pods` will return *"Metrics API not available"* and HPA will show `<unknown>/50%` for CPU until metrics-server is installed and running.

TERMINAL — INSTALL METRICS-SERVER ON DOCKER DESKTOP copy

```
# Option A: Official manifest with kubelet insecu
kubectl apply -f https://github.com/kubernetes-si

# Docker Desktop uses a self-signed cert — patch
kubectl patch deployment metrics-server -n kube-s
  --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/co

# Wait for it to be ready (~30 seconds)
kubectl rollout status deployment/metrics-server

# Verify it works — you should see CPU and Memory
kubectl top nodes
kubectl top pods
```

TERMINAL — OPTION B: HELM INSTALL (IF YOU HAVE HELM) copy

```
helm repo add metrics-server https://kubernetes-s
helm repo update
helm upgrade --install metrics-server metrics-ser
  --namespace kube-system \
  --set args={--kubelet-insecure-tls}
```

💡 **Where does metrics-server run?** It runs as a pod in the `kube-system` namespace on the master/control-plane node. Verify with: `kubectl get pods -n kube-system | grep metrics`. You should see `metrics-server-xxxx 1/1 Running`.

Create HPA via command

TERMINAL copy

```
kubectl autoscale deployment express-deployment \
  --min=1 \
  --max=5 \
  --cpu-percent=50
```

Or as a YAML file (recommended)

HPA.YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: express-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: express-deployment    # must match depl
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50   # scale when C
```

HPA scaling flow

```
Traffic increases → CPU per pod goes above 50%
    ↓
HPA adds new pod (up to maxReplicas: 5)
    ↓
Requests spread across more pods
    ↓
CPU per pod drops — each pod handles less ✅
    ↓
```

```
Traffic drops → CPU goes low
    ↓
HPA removes extra pods (down to minReplicas: 1)
✅
```

⚠️ **resources.requests must be set** in your deployment for HPA to work. HPA calculates utilization as *current CPU / requested CPU × 100*. Without requests defined, HPA has no baseline to calculate from and will not scale.

TERMINAL copy

```
kubectl get hpa          # see current CPU%,
kubectl top pods         # real-time CPU and
```

14 Managing Secrets Locally

Store sensitive config (DB passwords, API keys, tokens) safely in your local cluster

KEY CONCEPT

A **Secret** stores sensitive data like database passwords, API keys, and JWT secrets inside the Kubernetes cluster — encoded in base64. Pods then access these values as environment variables or file mounts. Unlike a ConfigMap (which is for non-sensitive config), Secrets signal to Kubernetes that this data should be handled with care (not logged, RBAC-restricted). In a local Docker Desktop setup secrets are fine as-is; in production you would use external vaults.

⚠️ **base64 is NOT encryption.** It is just encoding. Anyone with `kubectl get secret` access can decode it. Never commit secret YAML files to git. Use

`.gitignore` to exclude them, or use a tool like *sealed-secrets* or *external-secrets* for production.

Step 1 — Create base64 values

TERMINAL — ENCODE YOUR VALUES copy

```
# Encode any string to base64
echo -n 'mypassword' | base64
# → bXlwYXNzd29yZA==

echo -n 'mongodb://localhost:27017/mydb' | base64
# → bW9uZ29kYjovL2xvY2FsaG9zdDoyNzAxNy9teWRi

# Decode to verify
echo 'bXlwYXNzd29yZA==' | base64 --decode
```

Step 2 — Write secret.yaml

K8S/SECRET.YAML

```
apiVersion: v1
kind: Secret
metadata:
  name: express-secret
type: Opaque                # Opaque = generic ke
data:
  MONGO_URI: bW9uZ29kYjovL2xvY2FsaG9zdDoyNzAxNy9t
  JWT_SECRET: bXlzdXBlcnNlY3JldGtleQ==
  DB_PASSWORD: bXlwYXNzd29yZA==
```

💡 **Tip — use stringData instead of data** to write plain text directly (Kubernetes encodes it for you). This is easier for local development but the result is identical in the cluster.

K8S/SECRET.YAML — USING STRINGDATA (PLAINTEXT, EASIER LOCALLY)

```
apiVersion: v1
kind: Secret
metadata:
  name: express-secret
type: Opaque
stringData:                    # plain text — k8s b
  MONGO_URI: "mongodb://localhost:27017/mydb"
  JWT_SECRET: "mysupersecretkey"
  DB_PASSWORD: "mypassword"
```

Step 3 — Apply the Secret

TERMINAL copy

```
kubectl apply -f k8s/secret.yaml

# Verify it was created
kubectl get secrets
kubectl describe secret express-secret   # shows

# Decode a specific value to verify
kubectl get secret express-secret -o jsonpath='{.
```

Step 4 — Inject Secret into your Deployment as env vars

DEPLOYMENT.YAML — ENV SECTION

```
spec:
  containers:
    - name: express
      image: express-k8s:latest
      env:
        - name: MONGO_URI          # name of env
          valueFrom:
            secretKeyRef:
              name: express-secret # secret name
              key: MONGO_URI         # key inside
        - name: JWT_SECRET
          valueFrom:
```

```
            secretKeyRef:
              name: express-secret
              key: JWT_SECRET
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: express-secret
              key: DB_PASSWORD
```

Access in your Node.js/Express code

SERVER.JS

```
// Kubernetes injects secrets as environment vari
const mongoUri    = process.env.MONGO_URI;
const jwtSecret   = process.env.JWT_SECRET;
const dbPassword  = process.env.DB_PASSWORD;

// Same code works locally with .env and in Kuber
```

Secret vs ConfigMap — when to use which

Use case	ConfigMap	Secret
Database password	❌ Never	✅ Yes
API base URL	✅ Yes	Not needed
JWT secret key	❌ Never	✅ Yes
NODE_ENV = production	✅ Yes	Not needed
MongoDB connection string	❌ if it has password	✅ Yes
Port number	✅ Yes	Not needed

Add secret.yaml to your project structure

PROJECT LAYOUT (UPDATED)

```
your-project/
├── Backend/
│   ├── server.js
│   ├── package.json
│   └── dockerfile
└── k8s/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── hpa.yaml
    └── secret.yaml        # ← add to .gitignore !
```

⚠️ **Always add secret.yaml to .gitignore.** Committing secrets to git — even encoded base64 — is a serious security risk. Add `k8s/secret.yaml` to your `.gitignore` file. Share secrets with teammates via a secure channel (1Password, Bitwarden, private Slack DM), never via git.

Apply order matters

TERMINAL — ALWAYS APPLY SECRET BEFORE DEPLOYMENT copy

```
# 1. Secret must exist before pods that reference
kubectl apply -f k8s/secret.yaml

# 2. Then apply deployment (pods can now find the
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml

# Verify env vars were injected into the pod
kubectl exec -it <pod-name> -- sh
# Inside pod:
echo $MONGO_URI
echo $JWT_SECRET
```

Sheryians Coding School · Kubernetes Local Setup Notes

Docker Desktop · MERN Stack · nginx Ingress